

AUCTORUM

FASE 1 — ROADMAP TÁCTICO

De Asistente Reactivo a Agente Autónomo

Planificación Estratégica y Técnica

Versión 1.0 | 27 de Febrero de 2026

Arquitecto: Armando Javier Flores Salazar

Ingeniería en Tecnologías Computacionales | Tec de Monterrey
Proyecto Mindshare Oracle 2026a

1. Análisis Ejecutivo: Estado Actual y Visión de Fase 1

1.1 Dónde Estamos (Fase 0 — Completada)

AUCTORUM es un sistema reactivo funcional. Escucha, procesa, ejecuta. Tiene 6 modelos orquestados con hot-swap, controla Gmail, Calendar, GitHub, Spotify, responde por WhatsApp, y opera detrás de 7 capas de seguridad. Todo local, todo soberano.

Pero es reactivo. Espera órdenes. No se anticipa. No aprende de sus interacciones. No recuerda contexto entre sesiones más allá de archivos .md manuales. No monitorea su propia salud. Sus credenciales viven en texto plano en scripts.

1.2 A Dónde Vamos (Fase 1 — Autonomía Proactiva)

Fase 1 transforma a AUCTORUM de “asistente que espera” a “agente que se anticipa”. Los 5 pilares son:

1. **Rutinas Autónomas** — Resúmenes matutinos, alertas proactivas, tareas programadas sin intervención humana.
2. **Ciclo de Memoria** — Memoria momentánea, de mediano plazo y de largo plazo con persistencia real.
3. **Gestión de Secretos** — Vaultwarden para eliminar credenciales en texto plano.
4. **Seguridad Activa** — CrowdSec + Aegis como sistema de detección y respuesta a amenazas.
5. **Refinación de Agentes** — Optimización de prompts, reducción de latencia, mejora de delegación.

1.3 Restricciones Físicas Inmutables

Toda la planificación debe respetar estas limitaciones de hierro:

- **GPU:** GTX 1070, 8GB VRAM. No cambia.
- **CPU:** i3-7100, 2 cores / 4 threads. Los cronjobs compiten con Ollama por CPU.
- **RAM:** 16GB DDR4. ChromaDB y Qdrant consumen RAM que también necesita Ollama.
- **Red:** Solo Tailscale. Sin puertos públicos.
- **Orquestador:** OpenClaw 2026.2.23. Heartbeat existente pero limitado.

La regla de oro es: cada nuevo componente debe justificar su consumo de recursos contra el beneficio que aporta.

2. Pilar 1 — Rutinas Autónomas (CRON/Heartbeat)

2.1 Objetivo

Que Auctorum envíe un resumen matutino por WhatsApp sin que Armando lo pida. Contenido: clima, agenda del día, correos no leídos importantes.

2.2 Análisis de Viabilidad: Heartbeat de OpenClaw vs. Cronjobs de Linux

Tenemos dos opciones para disparar rutinas autónomas. El análisis honesto:

Opción A: Heartbeat de OpenClaw (Recomendada)

Ventajas:

- Ya existe. HEARTBEAT.md y WORKFLOW_AUTO.md están configurados.
- Opera dentro del contexto del agente — Qwen3 tiene acceso a su system prompt, herramientas exec, y el WhatsApp bridge de forma nativa.
- El mensaje se envía por WhatsApp directamente porque el agente ya está conectado al bridge.
- No requiere scripts externos que hablen con la API de WhatsApp por separado.

Desventajas:

- El heartbeat de OpenClaw no es un cron preciso. Se dispara periódicamente pero no a horas exactas.
- Si Qwen3 está ocupado con una petición del usuario, el heartbeat espera.
- No tenemos control fino del intervalo (depende de la configuración interna de OpenClaw).

Opción B: Cronjobs nativos de Linux

Ventajas:

- Precisión de reloj. `0 7 * * *` significa 7:00 AM exacto.
- Independiente del estado de Qwen3 o OpenClaw.

Desventajas:

- Un cronjob no tiene acceso nativo al WhatsApp bridge de OpenClaw. Tendría que enviar mensajes inyectando comandos en la API de OpenClaw o usando un script separado — esto añade complejidad y puntos de fallo.
- Si el cronjob dispara una inferencia con Qwen3 mientras otro mensaje está siendo procesado, puede haber colisiones de VRAM.
- Requiere mantener dos sistemas de automatización en paralelo.

Veredicto

Usar el Heartbeat de OpenClaw como mecanismo primario, mejorado con un cronjob de soporte que solo escribe un archivo trigger.

El flujo es: un cronjob escribe a las 7:00 AM un archivo

`~/openclaw/workspace/MORNING_TRIGGER.md` con la fecha actual. El heartbeat de OpenClaw, en su próxima ejecución (que debería ser dentro de pocos minutos), lee este archivo, detecta que

hay un trigger pendiente, ejecuta la rutina matutina, y borra el trigger. Así obtenemos la precisión del cron con la integración nativa del heartbeat.

2.3 Diseño del Resumen Matutino

Script: morning_routine.sh

Este script será invocado por Qwen3 cuando detecte el trigger matutino. Ejecuta las 3 consultas en secuencia y retorna un bloque de texto formateado para WhatsApp.

```
#!/bin/bash
# ~/.openclaw/workspace/morning_routine.sh
# Rutina matutina — invocado por Qwen3 via heartbeat

echo "* *RESUMEN MATUTINO — $(date '+%A %d de %B')*"
echo ""

# 1. Clima
echo "🌤️ *Clima en Saltillo:*"
curl -s "wttr.in/Saltillo?format=%C+%t+%h+%w" 2>/dev/null || echo "No disponible"
echo ""

# 2. Agenda del día
echo "📅 *Agenda de hoy:*"
gog calendar list --from today --to today 2>/dev/null || echo "Sin eventos"
echo ""

# 3. Correos importantes no leídos
echo "✉️ *Correos no leídos importantes:*"
gog gmail search "is:unread is:important newer_than:1d" --max 5 2>/dev/null || echo "Sin co"
echo ""
echo "---"
echo "_Generado automáticamente por Auctorum_"
```

Cronjob trigger

```
# crontab -e (usuario cocopsn)
0 7 * * * echo "MORNING_$(date +%Y-%m-%d)" > /home/cocopsn/.openclaw/workspace/MORNING_TRIGGER.md
```

Actualización requerida en AGENTS.md

Agregar a la sección HEARTBEAT una instrucción:

```
Durante cada heartbeat, verifica si existe el archivo MORNING_TRIGGER.md.
Si existe y contiene la fecha de hoy:
1. Ejecuta: exec -> bash ~/.openclaw/workspace/morning_routine.sh
2. Envía el resultado como mensaje de WhatsApp al usuario.
3. Elimina el archivo trigger: exec -> rm ~/.openclaw/workspace/MORNING_TRIGGER.md
```

Rutinas adicionales planificadas

Rutina	Hora	Contenido
Resumen matutino	7:00 AM	Clima + agenda + correos
Check de mediodía	1:00 PM	Correos nuevos urgentes, eventos en 2h

Resumen nocturno	10:00 PM	Resumen del día, eventos de mañana
Health check	Cada 6h	Estado de Ollama, VRAM, disco, servicios

2.4 Primera Acción Técnica

1. Crear `morning_routine.sh` con permisos de ejecución.
2. Agregar el cronjob trigger.
3. Actualizar AGENTS.md con las instrucciones de heartbeat.
4. Probar manualmente creando el trigger y esperando el heartbeat.

2.5 Riesgos y Mitigaciones

Riesgo	Probabilidad	Mitigación
Heartbeat no detecta trigger a tiempo	Media	Configurar intervalo de heartbeat más corto
Qwen3 ocupado cuando llega el trigger	Baja	El trigger persiste hasta ser procesado
gogcli token expirado	Media	Validar token en <code>morning_routine.sh</code> antes de consultar
Mensaje demasiado largo para WhatsApp	Baja	Limitar a 5 correos y 10 eventos max

3. Pilar 2 — Ciclo de Memoria del Agente

3.1 Problema Actual

Qwen3:8B tiene una ventana de contexto de 16K tokens. Cuando la conversación termina, todo el contexto desaparece. Los archivos MEMORY.md y FECHA.md son un parche manual — el agente tiene que leerlos activamente, lo cual consume tokens de contexto y depende de que alguien (o el heartbeat) le recuerde hacerlo.

El resultado es un agente con amnesia crónica. Cada sesión empieza desde cero.

3.2 Arquitectura de Memoria de 3 Niveles

Nivel 1: Memoria Momentánea (Ventana de Contexto)

Qué es: Los 16,384 tokens del contexto activo de Qwen3.

Duración: La sesión actual. Se pierde al terminar.

Acción requerida: Ninguna — ya funciona. Pero debemos optimizar qué se inyecta. Actualmente AGENTS.md consume ~4K tokens del contexto. Si agregamos memoria persistente, debemos ser selectivos con lo que se inyecta para no saturar la ventana.

Optimización propuesta: Crear un `AGENTS_COMPACT.md` que sea una versión reducida (~2K tokens) del system prompt, reservando más espacio para contexto conversacional y memoria inyectada.

Nivel 2: Memoria de Mediano Plazo (SQLite)

Qué es: Base de datos local que almacena resúmenes de conversaciones, acciones ejecutadas, y contexto relevante.

Duración: Días a semanas. Se acumula y se prune periódicamente.

Implementación propuesta:

```
-- ~/.openclaw/workspace/memory/auctorum_memory.db

CREATE TABLE conversations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp TEXT NOT NULL,
  channel TEXT NOT NULL,      -- 'whatsapp', 'pwa', 'heartbeat'
  user_message TEXT,
  agent_response TEXT,
  actions_taken TEXT,         -- JSON: comandos ejecutados
  models_used TEXT,           -- JSON: ['qwen3', 'glm4']
  summary TEXT,               -- Resumen generado por LLM
  tags TEXT,                  -- JSON: ['spotify', 'gmail', 'urgente']
  context_tokens_used INTEGER
);

CREATE TABLE facts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp TEXT NOT NULL,
  category TEXT NOT NULL,     -- 'preference', 'project', 'decision', 'personal'
  fact TEXT NOT NULL,
  source_conversation_id INTEGER,
  confidence REAL DEFAULT 1.0,
```

```
    expires_at TEXT,                -- NULL = permanente
    FOREIGN KEY (source_conversation_id) REFERENCES conversations(id)
);

CREATE TABLE daily_summaries (
    date TEXT PRIMARY KEY,
    summary TEXT NOT NULL,
    key_events TEXT,                -- JSON
    mood TEXT,                      -- detectado por el agente
    generated_at TEXT
);

CREATE INDEX idx_conv_timestamp ON conversations(timestamp);
CREATE INDEX idx_facts_category ON facts(category);
CREATE INDEX idx_facts_expires ON facts(expires_at);
```

Script de memoria: memory_manager.py

Este script Python será invocado por Qwen3 para leer y escribir memoria:

```
memory_manager.py save "Armando prefiere que no le pregunten antes de ejecutar"
memory_manager.py recall "preferencias de Armando"
memory_manager.py summarize_day
memory_manager.py context_for "spotify"
```

Flujo de integración:

- 1. Al inicio de cada conversación, el heartbeat inyecta los últimos 5 hechos relevantes y el resumen del día anterior.
- 2. Al final de cada conversación (o cada N mensajes), Qwen3 genera un resumen y lo guarda via `memory_manager.py save`.
- 3. Cada noche a las 11 PM, un trigger dispara `memory_manager.py summarize_day`.

Consumo de recursos: SQLite es virtualmente gratuito — cero VRAM, ~1MB de RAM, archivos en disco. Es la opción óptima para nuestro hardware.

Nivel 3: Memoria de Largo Plazo (Base Vectorial — Futuro)

Qué es: ChromaDB o Qdrant indexando documentos, notas, y conversaciones históricas para búsqueda semántica (RAG).

Análisis de viabilidad para nuestra GTX 1070:

ChromaDB necesita un modelo de embeddings para convertir texto en vectores. Las opciones:

Modelo de Embeddings	RAM	VRAM	Velocidad	Calidad
all-MiniLM-L6-v2	~100MB	0 (CPU)	Rápida	Buena
nomic-embed-text	~300MB	~1GB (GPU)	Rápida	Muy buena
BGE-small-en	~130MB	0 (CPU)	Rápida	Buena

Veredicto: Viable pero NO para Fase 1 inmediata. El modelo de embeddings corriendo en CPU (all-MiniLM-L6-v2) es gratuito en VRAM pero compite por CPU con Ollama. En GPU (nomic-embed-text), roba 1GB de los 8GB de VRAM, reduciendo el espacio disponible para Qwen3.

Plan: Implementar en Fase 1.5 una vez que la memoria SQLite esté operativa y probada. La base vectorial se construye sobre los datos ya estructurados en SQLite.

Integración con Obsidian/Logseq

Análisis honesto: Armando no reportó usar Obsidian o Logseq actualmente. Implementar un watcher de filesystem para indexar una bóveda que no existe es ingeniería prematura. Sin embargo, la arquitectura de memoria SQLite + futura base vectorial está diseñada para que agregar un indexador de Obsidian sea trivial cuando sea necesario.

Plan: Diferir a Fase 2. Si Armando adopta Obsidian para sus clases de ITC, el indexador se construye como un script Python que lee los .md y los inserta en la tabla `facts`.

3.3 Primera Acción Técnica

1. Crear el schema SQLite con las 3 tablas.
2. Escribir `memory_manager.py` con comandos `save/recall/summarize_day/context_for`.
3. Actualizar AGENTS.md para que Qwen3 use `memory_manager.py` al inicio y fin de cada sesión.
4. Agregar trigger nocturno para `summarize_day`.

3.4 Riesgos y Mitigaciones

Riesgo	Probabilidad	Mitigación
Base de datos crece sin control	Media	Prune automático: borrar conversaciones > 30 días, mantener solo resúmenes
Qwen3 no recuerda invocar <code>memory_manager</code>	Alta	Instrucción explícita en AGENTS.md + inyección automática en heartbeat
Resúmenes de baja calidad	Media	Usar DeepSeek-R1 para resúmenes (más lento pero mejor calidad)
Tokens de contexto consumidos por memoria	Media	Limitar inyección a 500 tokens de memoria por sesión

4. Pilar 3 — Gestión de Secretos (Vaultwarden)

4.1 Problema Actual

Las credenciales de AUCTORUM viven en texto plano en múltiples ubicaciones:

- **Spotify Client ID/Secret:** Hardcodeados en musica.sh.
- **Token de GitHub:** En ~/.config/gh/hosts.yml.
- **Token OAuth de Google:** En archivos de gogcli.
- **Token del gateway:** En openclaw.json.
- **Refresh token de Spotify:** En user_client_token.json.

Si alguien obtiene acceso al filesystem (incluso de solo lectura), tiene todas las credenciales.

4.2 Qué es Vaultwarden

Vaultwarden es una implementación open-source y ligera del servidor Bitwarden, escrita en Rust. Consume ~30MB de RAM y cero VRAM. Almacena credenciales cifradas con AES-256 y las expone via una API REST local.

4.3 Arquitectura Propuesta

```
[Auctorum (Qwen3)]
  → exec: vault_get.sh "spotify_client_id"
  → vault_get.sh consulta API local de Vaultwarden
  → Vaultwarden retorna el secreto cifrado
  → vault_get.sh lo descripta y retorna a stdout
  → Qwen3 usa el secreto en el comando
  → El secreto nunca se escribe en disco ni en logs
```

Script: vault_get.sh

```
#!/bin/bash
# Obtiene un secreto de Vaultwarden sin exponerlo
SECRET_NAME="$1"
BW_SESSION=$(cat ~/.bw_session 2>/dev/null)

if [ -z "$BW_SESSION" ]; then
    BW_SESSION=$(bw unlock --raw 2>/dev/null)
    echo "$BW_SESSION" > ~/.bw_session
    chmod 600 ~/.bw_session
fi

bw get password "$SECRET_NAME" --session "$BW_SESSION" 2>/dev/null
```

Migración de Credenciales

Credencial	Ubicación Actual	Destino en Vaultwarden
Spotify Client ID	musica.sh (hardcoded)	vault://spotify_client_id
Spotify Client Secret	musica.sh (hardcoded)	vault://spotify_client_secret
Spotify Refresh Token	user_client_token.json	vault://spotify_refresh

GitHub Token	~/config/gh/hosts.yml	vault://github_token
Google OAuth	gogcli config	vault://google_oauth
Gateway Token	openclaw.json	Se mantiene (OpenClaw lo gestiona)

4.4 Viabilidad en Nuestro Hardware

Recurso	Consumo de Vaultwarden	Disponible
RAM	~30MB	16GB - ~8GB (Ollama) = ~8GB libres
VRAM	0	N/A
CPU	Mínimo (solo en consultas)	Aceptable
Disco	~50MB (binario + datos)	Abundante
Puerto	8222 (local)	No conflict con 11434 ni 18789

Veredicto: Completamente viable. Cero impacto en rendimiento de inferencia.

4.5 Orden de Implementación

1. Instalar Vaultwarden como servicio systemd.
2. Crear vault_get.sh.
3. Migrar credenciales de Spotify primero (las más expuestas por estar hardcodeadas).
4. Modificar musica.sh para usar vault_get.sh en vez de constantes.
5. Migrar resto de credenciales gradualmente.

4.6 Riesgo Crítico

Si Vaultwarden cae, musica.sh y otros scripts dejan de funcionar. Mitigación: el script vault_get.sh tiene un fallback que lee las credenciales de un archivo local cifrado con GPG como respaldo de emergencia. No es ideal, pero garantiza que el sistema no se paralice.

5. Pilar 4 — Seguridad Activa (CrowdSec + Aegis)

5.1 Estado Actual de Seguridad

AUCTORUM tiene 7 capas de seguridad pasiva. Ninguna es activa — ninguna detecta amenazas en tiempo real ni responde a ellas. Aegis Sentinel clasifica prompts como SAFE/THREAT, pero no monitorea tráfico de red ni logs del sistema.

5.2 Qué es CrowdSec

CrowdSec es un IDS/IPS (Intrusion Detection/Prevention System) open source con un motor de comportamiento que analiza logs en tiempo real. Detecta:

- Escaneos de puertos
- Fuerza bruta SSH
- Patrones de tráfico malicioso
- IPs conocidas como maliciosas (base de datos comunitaria)

Cuando detecta una amenaza, puede bloquear la IP automáticamente via bouncers (plugins que se integran con UFW, iptables, nginx, etc.).

5.3 Arquitectura CrowdSec + Aegis

```
[Logs del sistema]
→ [CrowdSec Engine] (analiza journalctl, auth.log, nginx.log)
→ Detecta anomalía
→ Escribe alerta en /var/log/crowdsec/alerts.json

[Heartbeat de Auctorum]
→ Lee alerts.json periódicamente
→ Si hay alertas nuevas:
    → Invoca Aegis Sentinel con contexto de la alerta
    → Aegis clasifica severidad: INFO / WARNING / CRITICAL
    → Si WARNING+: Qwen3 notifica a Armando por WhatsApp
    → Si CRITICAL: Bloqueo automático de IP + notificación
    → Registro en memory/security_log.db
```

Script: security_monitor.sh

```
#!/bin/bash
# Monitoreo de alertas CrowdSec
ALERT_FILE="/var/log/crowdsec/alerts.json"
LAST_CHECK="$HOME/.openclaw/workspace/.last_security_check"

if [ ! -f "$ALERT_FILE" ]; then
    echo "NO_ALERTS"
    exit 0
fi

# Obtener alertas desde el último check
LAST_TS=$(cat "$LAST_CHECK" 2>/dev/null || echo "1970-01-01T00:00:00Z")
NEW_ALERTS=$(python3 -c "
import json, sys
from datetime import datetime
last = '$LAST_TS'
```

```
with open('$ALERT_FILE') as f:
    alerts = json.load(f)
new = [a for a in alerts if a.get('created_at','') > last]
if new:
    for a in new[-5:]: # últimas 5
        print(f"{a.get('scenario','unknown')} | IP: {a.get('source',{}).get('ip','?')} | s
    else:
        print('NO_ALERTS')
")

date -u +%Y-%m-%dT%H:%M:%SZ > "$LAST_CHECK"
echo "$NEW_ALERTS"
```

5.4 Consumo de Recursos

Componente	RAM	VRAM	CPU
CrowdSec Engine	~80MB	0	Bajo (picos en parsing)
CrowdSec Bouncer (UFW)	~10MB	0	Mínimo
security_monitor.sh	~5MB	0	Solo en ejecución
Aegis (ya existente)	~0.8GB VRAM	Ya cargado	Ya contabilizado

Total incremental: ~95MB RAM, 0 VRAM adicional. Viable.

5.5 Viabilidad Honesta

CrowdSec está diseñado para servidores expuestos al internet. AUCTORUM opera detrás de Tailscale sin puertos públicos. Esto significa que CrowdSec tendrá muy poco tráfico que analizar — la mayoría de las amenazas que detecta son escaneos de puertos y fuerza bruta SSH que llegan desde el internet público, que AUCTORUM no recibe.

Pero tiene valor defensivo en profundidad:

- 1. Si alguien compromete un dispositivo dentro del tailnet, CrowdSec detecta el tráfico anómalo interno.
- 2. La base de datos comunitaria de IPs maliciosas sirve como lista de bloqueo preventiva.
- 3. El monitoreo de logs de auth.log detecta intentos de escalación de privilegios locales.
- 4. Es un componente valioso para el portafolio de Armando — demuestra seguridad enterprise en hardware consumer.

Veredicto: Implementar, pero con prioridad menor que rutinas autónomas y memoria. Su valor inmediato es más demostrativo que funcional dado nuestro modelo de red.

5.6 Orden de Implementación

- 1. Instalar CrowdSec + bouncer UFW.
 - 2. Configurar parsers para auth.log y journalctl.
 - 3. Crear security_monitor.sh.
 - 4. Integrar en AGENTS.md como tarea de heartbeat cada 6 horas.
 - 5. Actualizar Aegis Sentinel para aceptar contexto de seguridad de red (no solo prompts).
-

6. Pilar 5 — Refinación de Agentes y Prompts

6.1 Problemas Actuales con los Agentes

Después de 5 días de operación, estos son los problemas observados con los agentes:

Qwen3:8B (MAIN)

- **Verbosidad excesiva:** A veces explica lo que va a hacer en vez de simplemente hacerlo. Desperdicia tokens de contexto.
- **Alucinación de sintaxis:** Cuando no tiene el comando exacto en AGENTS.md, inventa sintaxis plausible pero incorrecta.
- **Delegación inconsistente:** A veces delega a GLM-4 cuando podría ejecutar directamente, y viceversa.
- **Thinking mode:** thinking=off está configurado, pero el modelo a veces genera razonamiento interno que el usuario ve.

GLM-4:9B (TOOLS)

- **Genera explicaciones además de comandos:** A pesar del system prompt que dice “solo genera el comando”, a veces agrega texto explicativo.
- **Latencia de hot-swap:** El ciclo descarga-carga-inferencia-descarga tarda 40-70 segundos. El usuario espera demasiado.

Aegis Sentinel

- **Demasiado permisivo:** El threshold para THREAT es muy alto. Algunos prompts sutilmente maliciosos pasan como SAFE.
- **Sin contexto:** Analiza cada prompt aislado, sin historial. Un ataque distribuido en múltiples mensajes pasa desapercibido.

6.2 Optimizaciones Propuestas

Para Qwen3:8B

Regla de Acción Directa en AGENTS.md:

```
REGLA CARDINAL: NUNCA expliques lo que vas a hacer. HAZLO.  
- MAL: "Voy a buscar tus correos no leídos..."  
- BIEN: [ejecuta el comando y muestra resultados]  
  
Si el comando está en AVAILABLE TOOLS, ejecútalo directamente.  
Si NO está, delega a GLM-4 via tool_executor.py.  
NO inventes sintaxis. Si no sabes el comando exacto, DELEGA.
```

Reducción de AGENTS.md:

El system prompt actual consume ~4K tokens. Propuesta:

- Mover la sección WHO IS ARMANDO a MEMORY.md (se inyecta solo cuando es relevante).
- Comprimir MULTI-MODEL ARCHITECTURE a una tabla de 5 líneas.
- Eliminar ejemplos redundantes.
- Objetivo: reducir a ~2.5K tokens, liberando 1.5K para memoria y contexto.

Para GLM-4:9B

System prompt más estricto:

```
Eres un traductor de lenguaje natural a comandos bash.
REGLAS:
1. Tu respuesta es EXACTAMENTE un comando bash. Nada más.
2. Sin explicaciones. Sin comentarios. Sin markdown.
3. Si no puedes generar el comando, responde: ERROR_UNKNOWN_COMMAND
4. Ejemplo de respuesta correcta: ~/.openclaw/workspace/musica.sh buscar "Sparks Coldplay"
5. Ejemplo de respuesta INCORRECTA: Para buscar la canción, ejecuta: `~/.openclaw/workspace
```

Para Aegis

Inyección de historial reciente:

Modificar aegis_sentinel.py para recibir los últimos 3 mensajes como contexto, no solo el mensaje actual. Esto detecta ataques distribuidos.

```
# Cambio en aegis_sentinel.py
# Antes: analyze(current_message)
# Después: analyze(last_3_messages + current_message)
```

Reducción de Latencia Global

Optimización	Ahorro Estimado	Complejidad
Reducir sleep de hot-swap de 3s a 1.5s	~1.5s por delegación	Baja
Cache de comandos frecuentes en AGENTS.md	Elimina delegación a GLM-4	Baja
Pre-load de Aegis en VRAM permanente	Elimina carga en cada análisis	Ya implementado
AGENTS.md compactado	~2s menos de procesamiento	Media
Respuesta sin explicación	~5s menos por respuesta	Baja

Ahorro total estimado: 10-15 segundos por interacción típica.

6.3 Inventario de Comandos Frecuentes para Cache

Estos comandos deberían estar directamente en AGENTS.md para evitar delegación a GLM-4:

```
musica.sh buscar "<canción artista>"
musica.sh next / prev / play / pause / toggle / info
gog gmail search "<filtro>" --max N
gog gmail read <id>
gog calendar list --from today --to tomorrow
gog calendar create "<título>" --start "<fecha>"
gh issue list --repo auctorum-project/auctorum
curl -s "wttr.in/Salttillo?format=3"
```

Si Qwen3 tiene estos 8 patrones memorizados, el 80% de las peticiones se ejecutan directamente sin hot-swap, reduciendo la latencia de 40-70 segundos a 10-15 segundos.

7. Evaluación de Capas Futuras (Fase 2+)

7.1 Base de Datos Vectorial Local (ChromaDB/Qdrant)

Análisis Técnico

Aspecto	ChromaDB	Qdrant
Lenguaje	Python	Rust
RAM base	~200MB	~100MB
Requiere GPU	Opcional	Opcional
API	Python nativa	REST + gRPC
Persistencia	SQLite backend	Propio
Complejidad	Baja	Media
Comunidad	Grande	Creciente

Recomendación: ChromaDB por simplicidad y porque todo nuestro stack de scripts es Python.

Modelo de Embeddings

Para nuestra GTX 1070, el modelo ideal es `all-MiniLM-L6-v2` : - Corre en CPU (no compite por VRAM) - 384 dimensiones (eficiente en storage) - ~100MB de RAM - Calidad suficiente para búsqueda en documentos personales

Flujo RAG Propuesto

```
[Usuario pregunta algo]
  → Qwen3 detecta que necesita contexto histórico
  → exec: rag_search.py "query del usuario"
  → rag_search.py busca en ChromaDB los 5 fragmentos más relevantes
  → Retorna los fragmentos como texto
  → Qwen3 inyecta los fragmentos en su contexto
  → Genera respuesta enriquecida con memoria
```

Impacto en VRAM

```
Estado normal: [Qwen3 ~6GB] [Aegis ~0.8GB] = 6.8GB
Con embeddings en CPU: Sin cambio en VRAM
Con embeddings en GPU: [Qwen3 ~6GB] [nomic ~1GB] = 7GB (ajustado)
```

Veredicto: Implementar en Fase 1.5 con embeddings en CPU. Cero impacto en VRAM.

7.2 Integración Obsidian/Logseq

Análisis

El valor de esta integración depende enteramente de que Armando adopte una de estas herramientas. El sistema de memoria SQLite + futura base vectorial está diseñado para que agregar un watcher de filesystem sea un script Python de ~50 líneas.

Implementación Futura

```
# obsidian_indexer.py (concepto)
import os, time, hashlib
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class VaultHandler(FileSystemEventHandler):
    def on_modified(self, event):
        if event.src_path.endswith('.md'):
            index_file(event.src_path)

def index_file(path):
    with open(path) as f:
        content = f.read()
    # Insertar en ChromaDB con metadata
    collection.add(
        documents=[content],
        metadatas=[{"source": path, "type": "obsidian"}],
        ids=[hashlib.md5(path.encode()).hexdigest()]
    )
```

Verdicto: Fase 2. Requiere que Armando adopte Obsidian y que ChromaDB esté operativo.

7.3 Pipeline de Voz (STT + TTS)

Análisis de Viabilidad

Componente	Modelo	VRAM	RAM	Calidad
STT (Speech-to-Text)	Whisper.cpp (tiny/base)	0 (CPU)	~200MB	Buena para español
TTS (Text-to-Speech)	Piper TTS	0 (CPU)	~100MB	Natural
Alternativa TTS	Coqui TTS	~1GB GPU	~500MB	Muy natural

Verdicto: Viable en CPU con Whisper.cpp + Piper TTS. Cero VRAM. Pero requiere micrófono y altavoz conectados al servidor o un flujo via WhatsApp voice messages. Fase 2.

7.4 Home Assistant

Análisis

Home Assistant corre como un contenedor Docker que consume ~500MB RAM. Se comunica con dispositivos IoT via WiFi/Zigbee/Z-Wave. La integración con AUCTORUM sería via la REST API de Home Assistant.

Prerequisito: Armando necesita dispositivos IoT (bombillas inteligentes, sensores, etc.). Sin dispositivos, no hay nada que controlar.

Verdicto: Fase 3. Requiere hardware IoT adicional.

8. Roadmap Táctico: Orden de Ejecución

8.1 Priorización por Impacto/Esfuerzo

Tarea	Impacto	Esfuerzo	Prioridad
Rutina matutina (cron + heartbeat)	Alto	Bajo	P0 — Inmediata
Refinación de AGENTS.md	Alto	Bajo	P0 — Inmediata
Cache de comandos frecuentes	Alto	Bajo	P0 — Inmediata
Memory SQLite + memory_manager.py	Alto	Medio	P1 — Semana 1
morning_routine.sh completo	Medio	Bajo	P1 — Semana 1
Optimización de system prompts	Medio	Bajo	P1 — Semana 1
Reducir sleep de hot-swap	Medio	Bajo	P1 — Semana 1
Vaultwarden instalación	Medio	Medio	P2 — Semana 2
Migración de credenciales a Vault	Medio	Medio	P2 — Semana 2
CrowdSec instalación	Bajo-Medio	Medio	P3 — Semana 3
CrowdSec + Aegis integración	Medio	Alto	P3 — Semana 3
ChromaDB + RAG	Alto	Alto	P4 — Semana 4+
Aegis con historial	Bajo	Medio	P4 — Semana 4+

8.2 Sprint 0 (Esta Semana — Ganancias Rápidas)

Día 1:

1. Crear `morning_routine.sh` y probarlo manualmente.
2. Agregar cronjob trigger para 7:00 AM.
3. Reducir AGENTS.md: mover WHO IS ARMANDO a memoria, comprimir tablas.
4. Agregar cache de los 8 comandos frecuentes directamente en AGENTS.md.

Día 2:

5. Crear schema SQLite de memoria.
6. Escribir `memory_manager.py` con save/recall/summarize_day.
7. Actualizar AGENTS.md para invocar memory_manager al inicio/fin de sesión.
8. Reducir sleep de hot-swap de 3s a 1.5s en los 4 scripts.

Día 3:

9. Probar rutina matutina completa end-to-end.
10. Probar ciclo de memoria: salvar hecho, recuperar hecho, generar resumen.
11. Verificar que la reducción de AGENTS.md no rompe comportamiento.
12. Benchmark de latencia: medir tiempos antes y después de optimizaciones.

8.3 Sprint 1 (Semana 2 — Secretos y Seguridad)

1. Instalar Vaultwarden como systemd service.
2. Crear vault_get.sh con fallback.

3. Migrar credenciales de Spotify.
4. Modificar musica.sh para usar vault_get.sh.
5. Migrar resto de credenciales.

8.4 Sprint 2 (Semana 3 — Monitoreo)

1. Instalar CrowdSec + bouncer.
2. Crear security_monitor.sh.
3. Integrar en heartbeat.
4. Probar con tráfico simulado.

8.5 Sprint 3 (Semana 4+ — Memoria Avanzada)

1. Instalar ChromaDB.
 2. Configurar embeddings en CPU (all-MiniLM-L6-v2).
 3. Crear rag_search.py.
 4. Indexar conversaciones históricas de SQLite.
 5. Probar RAG end-to-end.
-

9. Primera Acción Técnica a Ejecutar en el Servidor

Basado en todo el análisis anterior, la primera acción técnica que maximiza impacto con mínimo riesgo es:

Acción: Crear morning_routine.sh + Optimizar AGENTS.md

```
# PASO 1: Crear el script de rutina matutina
cat > ~/.openclaw/workspace/morning_routine.sh << 'MREOF'
#!/bin/bash
echo "*RESUMEN MATUTINO — $(date +%A %d de %B)*"
echo ""
echo "☁️ *Clima:*"
curl -s "wttr.in/Salttillo?format=%C+%t+%h+%w" 2>/dev/null || echo "No disponible"
echo ""
echo "📅 *Agenda:*"
gog calendar list --from today --to today 2>/dev/null || echo "Sin eventos"
echo ""
echo "📧 *Correos importantes:*"
gog gmail search "is:unread is:important newer_than:1d" --max 5 2>/dev/null || echo "Inbox"
MREOF
chmod +x ~/.openclaw/workspace/morning_routine.sh

# PASO 2: Agregar cronjob
(crontab -l 2>/dev/null; echo "0 7 * * * echo MORNING_$(date +%Y-%m-%d) > /home/cocopsr

# PASO 3: Actualizar AGENTS.md con:
# - Sección HEARTBEAT actualizada con detección de trigger
# - Cache de 8 comandos frecuentes
# - Regla CARDINAL de acción directa
# - Compresión de secciones verbosas
```

Esta acción se ejecuta en menos de 10 minutos y produce un resultado visible mañana a las 7:00 AM: el primer resumen matutino autónomo de Auctorum.

10. Métricas de Éxito de Fase 1

Métrica	Valor Actual (Fase 0)	Objetivo (Fase 1)
Resúmenes autónomos/día	0	3 (mañana, mediodía, noche)
Latencia promedio WhatsApp	40-70s (con delegación)	15-25s (cache de comandos)
Credenciales en texto plano	5+	0 (Vaultwarden)
Hechos recordados entre sesiones	0	Ilimitados (SQLite)
Alertas de seguridad detectadas	0	Todas (CrowdSec)
Tokens de AGENTS.md	~4K	~2.5K
Delegaciones evitables a GLM-4	~80% de requests	<20%

11. Conclusión Arquitectónica

Fase 1 no es una revolución — es una evolución disciplinada. Cada pilar se construye sobre la infraestructura probada de Fase 0. No requiere hardware nuevo, no requiere servicios cloud, y respeta la limitación de 8GB de VRAM que define todo el sistema.

La secuencia es deliberada: primero autonomía (rutinas + memoria), luego seguridad (secretos + CrowdSec), luego inteligencia (RAG + refinación). Cada paso valida el anterior antes de agregar complejidad.

La primera acción se ejecuta hoy. El primer resultado visible llega mañana a las 7:00 AM.

AUCTORUM dejará de esperar órdenes.

— Fin del Roadmap Táctico de Fase 1 —

AUCTORUM — De la Reacción a la Anticipación

La primera acción se ejecuta hoy. El primer resultado llega mañana a las 7:00 AM.